

Conservative Safety Critics for Exploration

24.07.24

Kangyeon Kim



Control and
Intelligent Systems
Laboratory

1. Introduction

(0) Goal

Develop **safe exploration methods** for RL that similarly exhibit conservative behavior, erring on the side of caution in particularly dangerous settings, and **limiting the number of catastrophic failures**.

1. Introduction

(1) Motivation

Most of these approaches require additional assumptions.

- Assuming prior domain knowledge of the problem structure: *safe set of environment states, initial safe policy, system dynamics, etc.*
- Assuming a continuous safety cost function: *only the optimal policy is learned to be safe, and there are no safety constraint satisfactions during training.*

→ **Our method only assumes access to a sparse (e.g., binary) indicator for catastrophic failure, in the standard RL setting.**

1. Introduction

(2) Contributions

[CSC, Conservative Safety Critics]

- (a) learn a conservative estimate of how safe a state is and apply it for safe-exploration & policy updates
- (b) theoretically providing upper bounds on the probability of failures throughout training

* CSC reduce the rate of catastrophic failures by up to 50% over prior safe exploration methods of five separate simulated robotic control domains spanning manipulation, navigation, and locomotion.

2. Preliminaries

(1) Off-policy RL algorithms

- parametric Q-function $Q_{\theta}(s, a)$, and parametric policy $\pi_{\phi}(a|s)$.
- behavior policy: $\pi_{\beta}(\mathbf{a}|\mathbf{s})$
- discounted state distribution: $d^{\pi_{\beta}}(\mathbf{s})$
- dataset which is sampled from $d^{\pi_{\beta}}(\mathbf{s})\pi_{\beta}(\mathbf{a}|\mathbf{s})$: $\mathcal{D}$
- empirical behavior policy: $\hat{\pi}_{\beta}(\mathbf{a}|\mathbf{s}) := \frac{\sum_{\mathbf{s}, \mathbf{a} \in \mathcal{D}} \mathbf{1}[\mathbf{s}=\mathbf{s}, \mathbf{a}=\mathbf{a}]}{\sum_{\mathbf{s} \in \mathcal{D}} \mathbf{1}[\mathbf{s}=\mathbf{s}]}$

2. Preliminaries

(1) Off-policy RL algorithms

- Q-learning methods: train Q-function by the Bellman optimality operator.

$$\mathcal{B}^* Q(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim P(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [\max_{\mathbf{a}'} Q(\mathbf{s}', \mathbf{a}')]]$$

- Actor-Critic methods: evaluate policy by Bellman operator and improve policy by updating it towards actions that maximize the expected Q-value.

$$\mathcal{B}^\pi Q = r + \gamma \mathbb{E}_{\mathbf{s}' \sim T(\mathbf{s}'|\mathbf{s}, \mathbf{a}), \mathbf{a}' \sim \pi(\mathbf{a}'|\mathbf{s}')} [Q(\mathbf{s}', \mathbf{a}')]]$$

- Problem in offline RL with dataset $\mathcal{D}$:
hard to apply multi-step learning because **dataset typically does not contain all possible transitions.**

2. Preliminaries

(1) Off-policy RL algorithms (Problem in Offline RL)

- Solution in Actor-Critic methods: use empirical Bellman operator $\hat{\mathcal{B}}^\pi$ that only backs up a single sample.

$$\mathcal{B}^\pi Q = r + \gamma \mathbb{E}_{s' \sim T(s'|s, a), a' \sim \pi(a'|s')} [Q(s', a')]$$



$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \mathbb{E}_{s, a, s' \sim \mathcal{D}} \left[\left(\underbrace{(r(s, a) + \gamma \mathbb{E}_{a' \sim \hat{\pi}^k(a'|s')} [\hat{Q}^k(s', a')])}_{\text{empirical Bellman backup}} - Q(s, a) \right)^2 \right] \quad (\text{policy evaluation})$$

$$\hat{\pi}^{k+1} \leftarrow \arg \max_{\pi} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi^k(a|s)} [\hat{Q}^{k+1}(s, a)] \quad (\text{policy improvement})$$

2. Preliminaries

(1) Off-policy RL algorithms (Problem in Offline RL)

- Solution in Actor-Critic methods: use empirical Bellman operator $\hat{\mathcal{B}}^\pi$ that only backs up a single sample.

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \mathbb{E}_{\underline{s, a, s' \sim \mathcal{D}}} \left[\left((r(s, a) + \gamma \mathbb{E}_{\underline{a' \sim \hat{\pi}^k(a'|s')}} [\hat{Q}^k(s', a')]) - Q(s, a) \right)^2 \right] \text{ (policy evaluation)}$$

$$\hat{\pi}^{k+1} \leftarrow \arg \max_{\pi} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi^k(a|s)} [\hat{Q}^{k+1}(s, a)] \text{ (policy improvement)}$$



“Action distribution shift”

Q-function is trained only on actions sampled from the behavior policy (!= learned policy)

2. Preliminaries

(1) Off-policy RL algorithms (Problem in Offline RL)

- Solution in Actor-Critic methods: use empirical Bellman operator $\hat{\mathcal{B}}^\pi$ that only backs up a single sample.

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left((r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{a}' \sim \hat{\pi}^k(\mathbf{a}'|\mathbf{s}')} [\hat{Q}^k(\mathbf{s}', \mathbf{a}')]) - Q(\mathbf{s}, \mathbf{a}) \right)^2 \right] \text{ (policy evaluation)}$$

$$\hat{\pi}^{k+1} \leftarrow \arg \max_{\pi} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi^k(\mathbf{a}|\mathbf{s})} [\hat{Q}^{k+1}(\mathbf{s}, \mathbf{a})] \text{ (policy improvement)}$$



“Action distribution shift”

Q-function is trained only on actions sampled from the behavior policy (!= learned policy)



“Overestimation problem”

may be biased towards OOD actions with erroneously high Q-values since policy is trained to maximize Q-values.

2. Preliminaries

(2) CQL (Solution in Offline RL)

- In addition to training Q-functions via standard Bellman error, **CQL minimizes the expected Q-values under a particular distribution of actions**, $\mu(\mathbf{s}, \mathbf{a}) = d^{\pi^\beta}(\mathbf{s})\mu(\mathbf{a}|\mathbf{s})$, and maximizes the expected Q-value under the on-policy distribution.

2. Preliminaries

(2) CQL (Solution in Offline RL)

- In addition to training Q-functions via standard Bellman error, **CQL minimizes the expected Q-values under a particular distribution of actions**, $\mu(\mathbf{s}, \mathbf{a}) = d^{\pi_\beta}(\mathbf{s})\mu(\mathbf{a}|\mathbf{s})$, and maximizes the expected Q-value under the on-policy distribution.

standard Q function training does not query at unobserved states, but query at unseen actions



distribution which include unseen action, $\mu(\mathbf{a}|\mathbf{s})$



$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \underbrace{\alpha \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]}_{\text{positive alpha}} + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right], \quad (1)$$

2. Preliminaries

(2) CQL (Solution in Offline RL)

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \alpha \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right], \quad (1)$$



if we only interested in value function, add Q-value maximization term.

→ more tighter lower bound (but no pointwise lower bound) $\mathbb{E}_{\pi(\mathbf{a}|\mathbf{s})} [\hat{Q}^\pi(\mathbf{s}, \mathbf{a})] \leq V^\pi(\mathbf{s})$

→ seems like method to only focus on the affect of unseen actions.



$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \alpha \cdot \left(\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \right) + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right]. \quad (2)$$

2. Preliminaries

(2) CQL (Solution in Offline RL)

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \alpha \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right], \quad (1)$$



if we only interested in value function, add Q-value maximization term.

→ **more tighter lower bound (but no pointwise lower bound)** $\mathbb{E}_{\pi(\mathbf{a}|\mathbf{s})} [\hat{Q}^\pi(\mathbf{s}, \mathbf{a})] \leq V^\pi(\mathbf{s})$

→ **seems like method to only focus on the affect of unseen actions.**



can be expressed as estimate for the advantage function, $A(\mathbf{s}, \mathbf{a})$

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \alpha \cdot \left(\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] \right) + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right]. \quad (2)$$

2. Preliminaries

(2) CQL (Solution in Offline RL)

Theorem 3.2 (Equation 2 results in a tighter lower bound). *The value of the policy under the Q -function from Equation 2, $\hat{V}^\pi(\mathbf{s}) = \mathbb{E}_{\pi(\mathbf{a}|\mathbf{s})}[\hat{Q}^\pi(\mathbf{s}, \mathbf{a})]$, lower-bounds the true value of the policy obtained via exact policy evaluation, $V^\pi(\mathbf{s}) = \mathbb{E}_{\pi(\mathbf{a}|\mathbf{s})}[Q^\pi(\mathbf{s}, \mathbf{a})]$, when $\mu = \pi$, according to:*

$$\forall \mathbf{s} \in \mathcal{D}, \quad \hat{V}^\pi(\mathbf{s}) \leq V^\pi(\mathbf{s}) - \alpha \left[(I - \gamma P^\pi)^{-1} \mathbb{E}_\pi \left[\frac{\pi}{\hat{\pi}_\beta} - 1 \right] \right] (\mathbf{s}) + \left[(I - \gamma P^\pi)^{-1} \frac{C_{r,T,\delta} R_{\max}}{(1 - \gamma) \sqrt{|\mathcal{D}|}} \right] (\mathbf{s}).$$

Thus, if $\alpha > \frac{C_{r,T} R_{\max}}{1 - \gamma} \cdot \max_{\mathbf{s} \in \mathcal{D}} \frac{1}{|\sqrt{|\mathcal{D}(\mathbf{s})|}|} \cdot \left[\sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}) \left(\frac{\pi(\mathbf{a}|\mathbf{s})}{\hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} - 1 \right) \right]^{-1}$, $\forall \mathbf{s} \in \mathcal{D}$, $\hat{V}^\pi(\mathbf{s}) \leq V^\pi(\mathbf{s})$, with probability $\geq 1 - \delta$. When $\hat{\mathcal{B}}^\pi = \mathcal{B}^\pi$, then any $\alpha > 0$ guarantees $\hat{V}^\pi(\mathbf{s}) \leq V^\pi(\mathbf{s})$, $\forall \mathbf{s} \in \mathcal{D}$.

In main paper, we obtain upper bound of safety critic value by reversing the sign of alpha!

2. Preliminaries

(3) CMDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma, \mu, \mathcal{C})$

- reward: $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$
- starting state distribution: μ
- set of safety constraint: $\mathcal{C} = \{(c_i : \mathcal{S} \rightarrow \{0, 1\}, \chi_i \in \mathbb{R}) | i \in \mathbb{Z}\}$
- stochastic policy: $\pi : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$
- discounted state distribution:

$$d^\pi(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi) \xrightarrow{\pi_\phi} \rho_\phi(s)$$

- State value function & state-action value function:

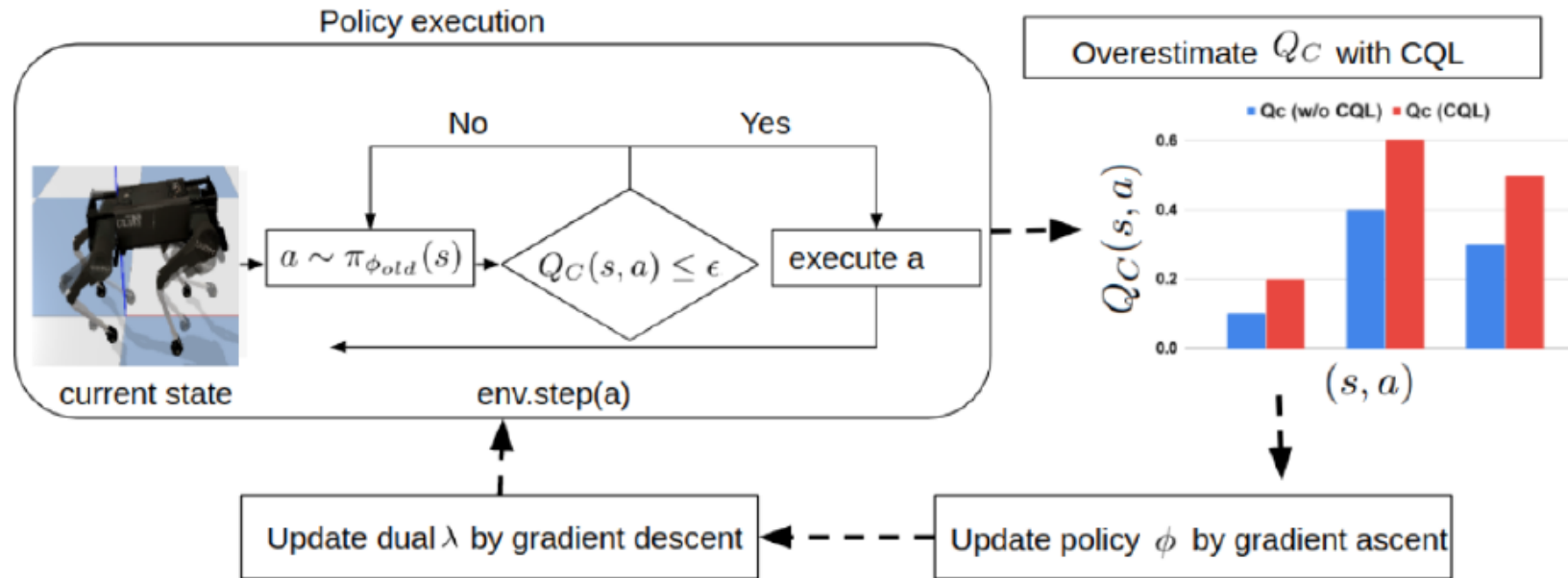
$$V_R^\pi(\mu) = \mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)]$$

$$\underline{V_C^\pi(\mu)} = \mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t C(s_t)] = \mathbb{E}_{\tau \sim \pi} [\mathbb{1}\{failure\}] = \mathbb{P}(failure | \mu)$$

probability of failure

3. Framework

(1) CSC algorithm



- Key Idea:
train Q_C that overestimates how unsafe a particular state is and modifies the exploration strategy.

3. Framework

(1) CSC algorithm

Algorithm 1 CSC: safe exploration with conservative safety critics

- 1: Initialize V_θ^r (task value fn), Q_ζ^s (safety critic), policy π_ϕ , λ , $\mathcal{D}_{env}$, thresholds ϵ, δ, χ .
 - 2: Set $\hat{V}_C^{\pi\phi_{old}}(\mu) \leftarrow \chi$. $\triangleright \hat{V}_C^{\pi\phi_{old}}(\mu)$ denotes avg. failures in the *previous* epoch.
 - 3: **for** epochs until convergence **do** $\triangleright$ Execute actions in the environment. Collect on-policy samples.
 - 4: **for** episode e in $\{1, \dots, M\}$ **do**
 - 5: Set $\epsilon \leftarrow (1 - \gamma)(\chi - \hat{V}_C^{\pi\phi_{old}}(\mu))$
 - 6: Sample $a \sim \pi_{\phi_{old}}(s)$. Execute a iff $Q_C(s, a) \leq \epsilon$. Else, resample a .
 - 7: Obtain next state $s', r = R(s, a), c = C(s')$.
 - 8: $\mathcal{D}_{env} \leftarrow \mathcal{D}_{env} \cup \{(s, a, s', r, c)\}$ $\triangleright$ If available, $\mathcal{D}_{env}$ can be seeded with off-policy/offline data
 - 9: **end for**
 - 10: Store the average episodic failures $\hat{V}_C^{\pi\phi_{old}}(\mu) \leftarrow \sum_{e=1}^M \hat{V}_C^e$
 - 11: **for** step t in $\{1, \dots, N\}$ **do** $\triangleright$ Policy and Q function updates using $\mathcal{D}_{env}$
 - 12: Gradient ascent on ϕ and (Optionally) add Entropy regularization (Appendix A.2)
 - 13: Gradient updates for the Q-function $\zeta := \zeta - \eta_Q \nabla_\zeta CQL(\zeta)$
 - 14: Gradient descent step on Lagrange multiplier λ (Appendix A.2)
 - 15: **end for**
 - 16: $\phi_{old} \leftarrow \phi$
 - 17: **end for**
-

3. Framework

(1) CSC algorithm

Algorithm 1 CSC: safe exploration with conservative safety critics

```
1: Initialize  $V_\theta^r$  (task value fn),  $Q_\zeta^s$  (safety critic), policy  $\pi_\phi$ ,  $\lambda$ ,  $\mathcal{D}_{env}$ , thresholds  $\epsilon, \delta, \chi$ .
2: Set  $\hat{V}_C^{\pi\phi_{old}}(\mu) \leftarrow \chi$ .  $\triangleright \hat{V}_C^{\pi\phi_{old}}(\mu)$  denotes avg. failures in the previous epoch.
3: for epochs until convergence do  $\triangleright$  Execute actions in the environment. Collect on-policy samples.
4:   for episode  $e$  in  $\{1, \dots, M\}$  do
5:     Set  $\epsilon \leftarrow (1 - \gamma)(\chi - \hat{V}_C^{\pi\phi_{old}}(\mu))$ 
6:     Sample  $a \sim \pi_{\phi_{old}}(s)$ . Execute  $a$  iff  $Q_C(s, a) \leq \epsilon$ . Else, resample  $a$ .
7:     Obtain next state  $s', r = R(s, a), c = C(s')$ .
8:      $\mathcal{D}_{env} \leftarrow \mathcal{D}_{env} \cup \{(s, a, s', r, c)\}$   $\triangleright$  If available,  $\mathcal{D}_{env}$  can be seeded with off-policy/offline data
9:   end for
10:  Store the average episodic failures  $\hat{V}_C^{\pi\phi_{old}}(\mu) \leftarrow \sum_{e=1}^M \hat{V}_C^e$ 
11:  for step  $t$  in  $\{1, \dots, N\}$  do  $\triangleright$  Policy and Q function updates using  $\mathcal{D}_{env}$ 
12:    Gradient ascent on  $\phi$  and (Optionally) add Entropy regularization (Appendix A.2)
13:    Gradient updates for the Q-function  $\zeta := \zeta - \eta_Q \nabla_\zeta CQL(\zeta)$ 
14:    Gradient descent step on Lagrange multiplier  $\lambda$  (Appendix A.2)
15:  end for
16:   $\phi_{old} \leftarrow \phi$ 
17: end for
```

* 실제 실험에서는 rejection sampling 과정은 100 iterations 동안 진행하고, threshold를 만족하는 값이 없으면 minimum에 해당하는 action을 사용했다고 한다.

3. Framework

(2) CSC algorithm (details)

- How to update the Q_C ?

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \alpha \cdot (\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \hat{\pi}_\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]) + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim \mathcal{D}} \left[\left(Q(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) \right)^2 \right]. \quad (2)$$

in CQL

$$\hat{Q}_C^{k+1} \leftarrow \arg \min_{Q_C} \alpha \cdot \left(-\mathbb{E}_{\mathbf{s} \sim \mathcal{D}_{env}, \mathbf{a} \sim \pi_\phi(\mathbf{a}|\mathbf{s})} [Q_C(\mathbf{s}, \mathbf{a})] + \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \mathcal{D}_{env}} [Q_C(\mathbf{s}, \mathbf{a})] \right) + \frac{1}{2} \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{s}', \mathbf{c}) \sim \mathcal{D}_{env}} \left[\left(Q_C(\mathbf{s}, \mathbf{a}) - \hat{\mathcal{B}}^{\pi_\phi} \hat{Q}_C^k(\mathbf{s}, \mathbf{a}) \right)^2 \right] \quad (2)$$

policy from safety-critic *can include off-policy data or offline data*

in CSC

3. Framework

(2) CSC algorithm (details)

- How to update the Q_C ?

maximize over actions sample from the current policy, while minimize over actions sample from the replay buffer

$$\hat{Q}_C^{k+1} \leftarrow \arg \min_{Q_C} \alpha \cdot \left(-\mathbb{E}_{s \sim \mathcal{D}_{env}, a \sim \pi_\phi(a|s)} [Q_C(s, a)] + \mathbb{E}_{(s,a) \sim \mathcal{D}_{env}} [Q_C(s, a)] \right) + \frac{1}{2} \mathbb{E}_{(s,a,s',c) \sim \mathcal{D}_{env}} \left[\left(Q_C(s, a) - \hat{\mathcal{B}}^{\pi_\phi} \hat{Q}_C^k(s, a) \right)^2 \right] \quad (2)$$

in CSC

3. Framework

(2) CSC algorithm (details)

- How to update the Q_C ?

$$-\hat{A}_C(s, a)$$

CQL-similar method can help provide an upper bound on the advantage function directly!



maximize over actions sample from the current policy, while minimize over actions sample from the replay buffer

$$\hat{Q}_C^{k+1} \leftarrow \arg \min_{Q_C} \alpha \cdot \left(-\mathbb{E}_{s \sim \mathcal{D}_{env}, a \sim \pi_\phi(a|s)} [Q_C(s, a)] + \mathbb{E}_{(s,a) \sim \mathcal{D}_{env}} [Q_C(s, a)] \right) + \frac{1}{2} \mathbb{E}_{(s,a,s',c) \sim \mathcal{D}_{env}} \left[\left(Q_C(s, a) - \hat{\mathcal{B}}^{\pi_\phi} \hat{Q}_C^k(s, a) \right)^2 \right] \quad (2)$$

in CSC

3. Framework

(2) CSC algorithm (details)

- How to update the Q_C ?

$$\begin{aligned}\hat{Q}_C^{k+1} \leftarrow \arg \min_{Q_C} & \quad \alpha \cdot \left(-\mathbb{E}_{s \sim \mathcal{D}_{env}, a \sim \pi_\phi(a|s)} [Q_C(s, a)] + \mathbb{E}_{(s,a) \sim \mathcal{D}_{env}} [Q_C(s, a)] \right) \\ & + \frac{1}{2} \mathbb{E}_{(s,a,s',c) \sim \mathcal{D}_{env}} \left[\left(Q_C(s, a) - \hat{\mathcal{B}}^{\pi_\phi} \hat{Q}_C^k(s, a) \right)^2 \right]\end{aligned}\tag{2}$$

3. Framework

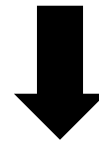
(2) CSC algorithm (details)

- How to update the policy?

[overall objective]

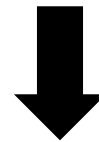
$$\pi^* = \arg \max_{\pi \in \Pi_C} V_R^\pi(\mu) \quad \text{where} \quad \Pi_C = \{\pi \in \Pi : V_C^\pi(\mu) \leq \underline{\chi}\} \quad (1)$$

probability of failure



equivalent to expectation of state action value

$$\max_{\underline{\pi}_\phi} \mathbb{E}_{s \sim \rho_\phi, a \sim \pi_\phi} [A_R^{\pi_\phi}(s, a)] \quad \text{s.t.} \quad \mathbb{E}_{s \sim \rho_\phi, a \sim \pi_\phi} Q_C(s, a) \leq \chi \quad (3)$$



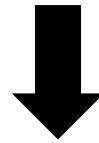
$$\max_{\pi_\phi} \min_{\lambda \geq 0} \mathbb{E}_{s \sim \rho_\phi, a \sim \pi_\phi} [A_R^{\pi_\phi}(s, a) - \lambda (Q_C(s, a) - \chi)]$$

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

$$\pi^* = \arg \max_{\pi \in \Pi_C} V_R^\pi(\mu) \quad \text{where} \quad \Pi_C = \{\pi \in \Pi : V_C^\pi(\mu) \leq \chi\} \quad (1)$$



$$\begin{aligned} & \max_{\pi_\phi} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} \left[A_R^{\pi_{\phi_{old}}} (s, a) \right] \\ & \text{s.t.} \quad \underline{\mathbb{E}_{s \sim \rho_{\phi_{old}}} [D_{\text{KL}}(\pi_{\phi_{old}}(\cdot|s) || \pi_\phi(\cdot|s))]} \leq \delta \quad \text{and} \quad V_C^{\pi_\phi}(\mu) \leq \chi \end{aligned} \quad (4)$$

“to ensure successive policies are close”

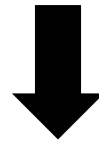
*in order to help obtain bounds on the expected failures of the new policy
in terms of the expected failures of the old policy.*

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

$$\pi^* = \arg \max_{\pi \in \Pi_C} V_R^\pi(\mu) \quad \text{where} \quad \Pi_C = \{\pi \in \Pi : V_C^\pi(\mu) \leq \chi\} \quad (1)$$



$$\begin{aligned} & \max_{\pi_\phi} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} \left[A_R^{\pi_{\phi_{old}}}(s, a) \right] \\ & \text{s.t.} \quad \mathbb{E}_{s \sim \rho_{\phi_{old}}} [D_{\text{KL}}(\pi_{\phi_{old}}(\cdot|s) || \pi_\phi(\cdot|s))] \leq \delta \quad \text{and} \quad V_C^{\pi_\phi}(\mu) \leq \chi \end{aligned} \quad (4)$$

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

Lemma 1. *If we follow Algorithm 1, during policy updates via Equation 5, the following is satisfied with high probability $\geq 1 - \omega$*

$$V_C^{\pi_{\phi_{old}}}(\mu) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi}} [A_C(s, a)] \leq \chi + \zeta - \frac{\Delta}{1-\gamma}$$

Here, ζ captures sampling error in the estimation of $V_C^{\pi_{\phi_{old}}}(\mu)$ and we have $\zeta \leq \frac{C' \sqrt{\log(1/\omega)}}{|N|}$, where C' is a constant independent of ω obtained from union bounds and concentration inequalities (Kumar et al., 2020) and N is the number of samples used in the estimation of V_C .

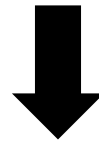
$$N \uparrow, \text{ then } \zeta = |\hat{V}_C^{\pi_{\phi_{old}}}(\mu) - V_C^{\pi_{\phi_{old}}}(\mu)| \downarrow$$

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

$$\begin{aligned} & \text{way to set epsilon} \\ Q_C(s, a) & \leq \underline{(1 - \gamma)(\chi - \hat{V}_C^{\pi_{\phi_{old}}(\mu)})} \quad \forall (s, a) \\ \implies \hat{A}_C(s, a) & \leq (1 - \gamma)(\chi - \hat{V}_C^{\pi_{\phi_{old}}(\mu)}) \quad \forall (s, a) \\ \implies \hat{V}_C^{\pi_{\phi_{old}}(\mu)} + \frac{1}{1 - \gamma} \hat{A}_C(s, a) & \leq \chi \quad \forall (s, a) \\ \implies \hat{V}_C^{\pi_{\phi_{old}}(\mu)} + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi}} [\hat{A}_C(s, a)] & \leq \chi \end{aligned} \tag{11}$$



If $\hat{V}_C^{\pi_{\phi_{old}}(\mu)} \geq V_C^{\pi_{\phi_{old}}(\mu)}$, then Lemma 1 hold directly (not with probability!)

3. Framework

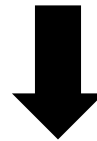
(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

way to set epsilon

$$\begin{aligned} Q_C(s, a) &\leq \underline{(1 - \gamma)(\chi - \hat{V}_C^{\pi_{\phi_{old}}(\mu)})} \quad \forall (s, a) \\ \implies \hat{A}_C(s, a) &\leq (1 - \gamma)(\chi - \hat{V}_C^{\pi_{\phi_{old}}(\mu)}) \quad \forall (s, a) \\ \implies \hat{V}_C^{\pi_{\phi_{old}}(\mu)} + \frac{1}{1 - \gamma} \hat{A}_C(s, a) &\leq \chi \quad \forall (s, a) \\ \implies \hat{V}_C^{\pi_{\phi_{old}}(\mu)} + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi}} [\hat{A}_C(s, a)] &\leq \chi \end{aligned} \tag{11}$$

Otherwise, use inequality which is related on zeta!



$$\zeta \leq \frac{C' \sqrt{\log(1/\omega)}}{|N|}$$

$$\begin{aligned} \hat{V}_C^{\pi_{\phi_{old}}(\mu)} + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi}} [\hat{A}_C(s, a)] &\leq \chi \\ \implies V_C^{\pi_{\phi_{old}}(\mu)} + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi}} [A_C(s, a)] &\leq \chi + \zeta - \frac{\Delta}{1 - \gamma} \end{aligned} \tag{12}$$

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

Lemma 1. *If we follow Algorithm 1, during policy updates via Equation 5, the following is satisfied with high probability $\geq 1 - \omega$*

$$V_C^{\pi_{\phi_{old}}}(\mu) + \frac{1}{1 - \gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi}} [A_C(s, a)] \leq \chi + \zeta - \frac{\Delta}{1 - \gamma}$$

Here, ζ captures sampling error in the estimation of $V_C^{\pi_{\phi_{old}}}(\mu)$ and we have $\zeta \leq \frac{C' \sqrt{\log(1/\omega)}}{|N|}$, where C' is a constant independent of ω obtained from union bounds and concentration inequalities (Kumar et al., 2020) and N is the number of samples used in the estimation of V_C .

Finally, we can bound the probability of failures. (using estimated from previous epoch)

However, Lemma 1 is only related on old policy!

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

$$\begin{aligned} \max_{\pi_\phi} \quad & \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} \left[A_R^{\pi_{\phi_{old}}} (s, a) \right] \\ \text{s.t.} \quad & \mathbb{E}_{s \sim \rho_{\phi_{old}}} [D_{\text{KL}}(\pi_{\phi_{old}}(\cdot|s) || \pi_\phi(\cdot|s))] \leq \delta \quad \text{and} \quad V_C^{\pi_\phi}(\mu) \leq \chi \end{aligned} \quad (4)$$

use additional constraint to establish Theorem 1, which also related on the new(updated) policy!

Theorem 1. Consider policy updates that solve the constrained optimization problem defined in equation 5. With high probability $\geq 1 - \omega$, we have the following upper bound on expected probability of failure $V_C^{\pi_{\phi_{new}}}(\mu)$ for $\pi_{\phi_{new}}$ during every policy update iteration

$$\underline{V_C^{\pi_{\phi_{new}}}(\mu)} \leq \chi + \zeta - \frac{\Delta}{1 - \gamma} + \frac{\sqrt{2\delta}\gamma\epsilon_C}{(1 - \gamma)^2} \quad \text{where} \quad \zeta \leq \frac{C\sqrt{\log(1/\omega)}}{|N|} \quad (13)$$

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

Conclusion from Lemma1:

We can satisfy the constraint in the objective during policy updates

Conclusion from Theorem1:

We can obtain an upper bound on the expected probability of failure of the update policy.

**** with high probability! (when we set sufficiently large sample number, N)***

3. Framework

(2) CSC algorithm (details)

Algorithm 1 CSC: safe exploration with conservative safety critics

```
1: Initialize  $V_\theta^r$  (task value fn),  $Q_\zeta^s$  (safety critic), policy  $\pi_\phi$ ,  $\lambda$ ,  $\mathcal{D}_{env}$ , thresholds  $\epsilon, \delta, \chi$ .
2: Set  $\hat{V}_C^{\pi\phi_{old}}(\mu) \leftarrow \chi$ .  $\triangleright \hat{V}_C^{\pi\phi_{old}}(\mu)$  denotes avg. failures in the previous epoch.
3: for epochs until convergence do  $\triangleright$  Execute actions in the environment. Collect on-policy samples.
4:   for episode  $e$  in  $\{1, \dots, M\}$  do
OK 5:     Set  $\epsilon \leftarrow (1 - \gamma)(\chi - \hat{V}_C^{\pi\phi_{old}}(\mu))$ 
6:     Sample  $a \sim \pi_{\phi_{old}}(s)$ . Execute  $a$  iff  $Q_C(s, a) \leq \epsilon$ . Else, resample  $a$ .
7:     Obtain next state  $s', r = R(s, a), c = C(s')$ .
8:      $\mathcal{D}_{env} \leftarrow \mathcal{D}_{env} \cup \{(s, a, s', r, c)\}$   $\triangleright$  If available,  $\mathcal{D}_{env}$  can be seeded with off-policy/offline data
9:   end for
OK 10: Store the average episodic failures  $\hat{V}_C^{\pi\phi_{old}}(\mu) \leftarrow \sum_{e=1}^M \hat{V}_C^e$ 
11:   for step  $t$  in  $\{1, \dots, N\}$  do  $\triangleright$  Policy and Q function updates using  $\mathcal{D}_{env}$ 
12:     Gradient ascent on  $\phi$  and (Optionally) add Entropy regularization (Appendix A.2)
13:     Gradient updates for the Q-function  $\zeta := \zeta - \eta_Q \nabla_\zeta CQL(\zeta)$ 
14:     Gradient descent step on Lagrange multiplier  $\lambda$  (Appendix A.2)
15:   end for
16:    $\phi_{old} \leftarrow \phi$ 
17: end for
```

Not yet,,

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

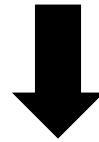
$$\begin{aligned} \max_{\pi_\phi} \quad & \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} \left[A_R^{\pi_{\phi_{old}}} (s, a) \right] \quad \text{s.t.} \quad \underline{V_C^{\pi_{\phi_{old}}}(\mu) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} [A_C(s, a)] \leq \chi} \\ \text{s.t.} \quad & \underline{\mathbb{E}_{s \sim \rho_{\phi_{old}}} [D_{\text{KL}}(\pi_{\phi_{old}}(\cdot|s) || \pi_\phi(\cdot|s))] \leq \delta} \end{aligned} \tag{5}$$

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**

$$\begin{aligned} \max_{\pi_\phi} \quad & \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} \left[A_R^{\pi_{\phi_{old}}}(s, a) \right] \quad \text{s.t.} \quad V_C^{\pi_{\phi_{old}}}(\mu) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} [A_C(s, a)] \leq \chi \\ \text{s.t.} \quad & \mathbb{E}_{s \sim \rho_{\phi_{old}}} [D_{\text{KL}}(\pi_{\phi_{old}}(\cdot|s) || \pi_\phi(\cdot|s))] \leq \delta \end{aligned} \quad (5)$$



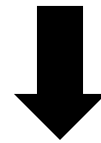
$$\begin{aligned} \max_{\pi_\phi} \min_{\lambda \geq 0} \quad & \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} \left[A_R^{\pi_{\phi_{old}}}(s, a) \right] - \lambda \left(V_C^{\pi_{\phi_{old}}}(\mu) + \frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_\phi} [\hat{A}_C(s, a)] - \chi \right) \\ \text{s.t.} \quad & \frac{1}{2}(\phi - \phi_{old})^T \mathbf{F}(\phi - \phi_{old}) \leq \delta \end{aligned} \quad (6)$$

second order term in the Taylor expansion of KL

3. Framework

(2) CSC algorithm (details)

- How to update the policy? **[Implementation details]**



Final calculation for gradient ascent

$$\phi \leftarrow \phi_{old} + \beta \mathbf{F}^{-1} \nabla_{\phi_{old}} \tilde{J}(\phi_{old}) \quad \beta = \sqrt{\frac{2\delta}{\nabla_{\phi_{old}} \tilde{J}(\phi_{old})^T \mathbf{F} \nabla_{\phi_{old}} \tilde{J}(\phi_{old})}} \quad (57)$$

$$\lambda \leftarrow \lambda - \left(\frac{1}{1-\gamma} \mathbb{E}_{s \sim \rho_{\phi_{old}}, a \sim \pi_{\phi_{old}}} [\hat{A}_C(s, a)] - \chi' \right) \quad (59)$$

3. Framework

(3) CSC algorithm (additional)

Theorem 2 (Convergence rate for policy gradient updates with the safety constraint). *If we run the policy gradient updates through equation 5, for policy π_ϕ , with μ as the starting state distribution, with $\phi^{(0)} = 0$, learning rate $\eta > 0$, and choose α as mentioned in the discussion of Theorem 1, then for all policy update iterations $T > 0$ we have, with probability $\geq 1 - \omega$,*

$$V_R^*(\mu) - V_R^{(T)}(\mu) \leq \frac{\log |\mathcal{A}|}{\eta T} + \frac{1}{(1 - \gamma)^2 T} + K \frac{\sum_{t=0}^{T-1} \lambda^{(t)}}{\eta T} \quad \text{where} \quad K \leq (1 - \chi) + \frac{4\sqrt{2\delta}\gamma}{(1 - \gamma)^2}$$

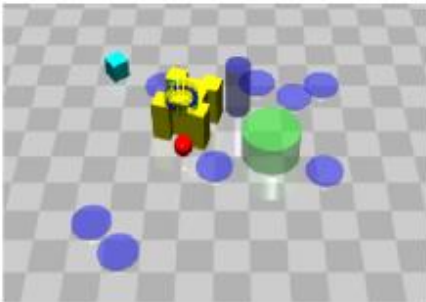
lower threshold $\rightarrow$ larger $K \rightarrow$ slower convergence

4. Experiments

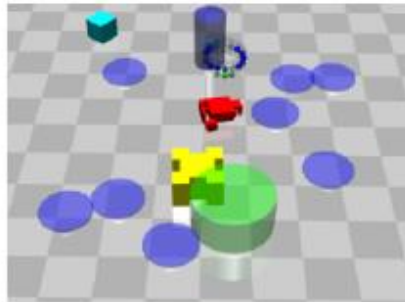
(1) Aim: empirically evaluate the agreement between empirical performance and theoretical guidance.

- How *safe* is CSC in terms of constraint satisfaction during training?
- How does learning of *safe policies* trade-off with task performance during training?

(2) Setup



(a) 2D nav.

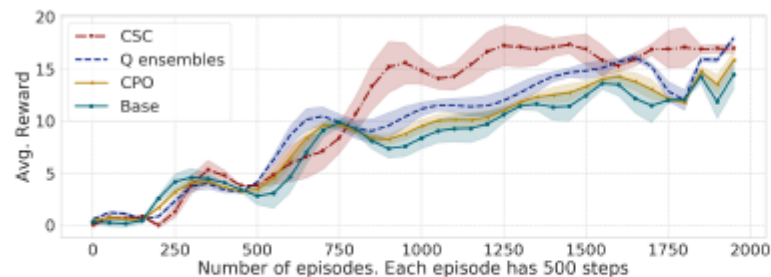


(b) Car nav.

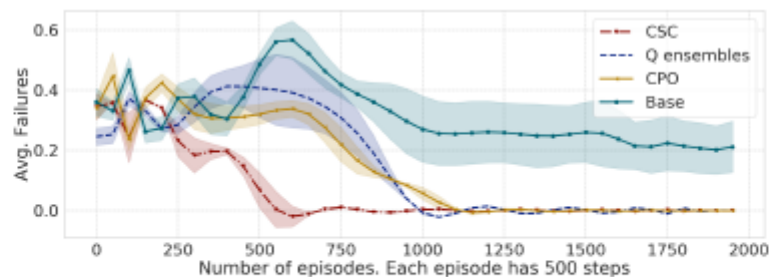
- health counter set 25 for the episode (500 steps).
- decrease by 1 for every time step in the trap.
- failure occurs when the counter strikes 0.

4. Experiments

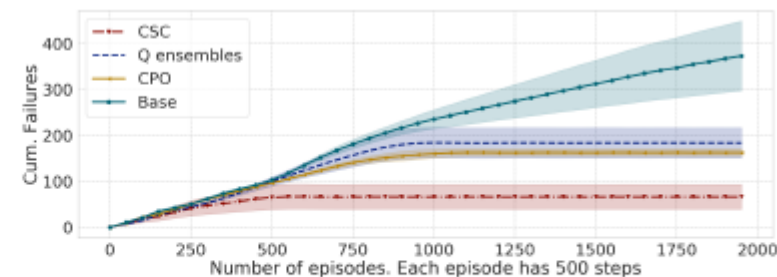
(3) Results



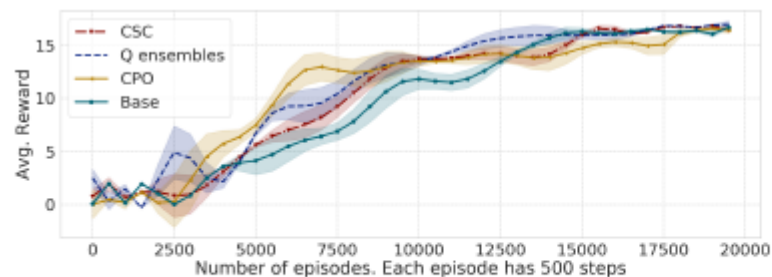
(a) Point 2D Nav. Rewards



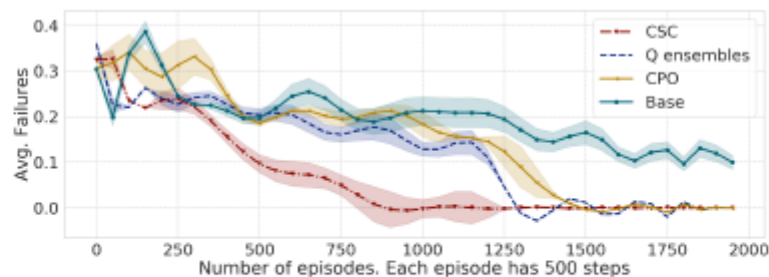
(b) Point 2D Nav. Avg. failures



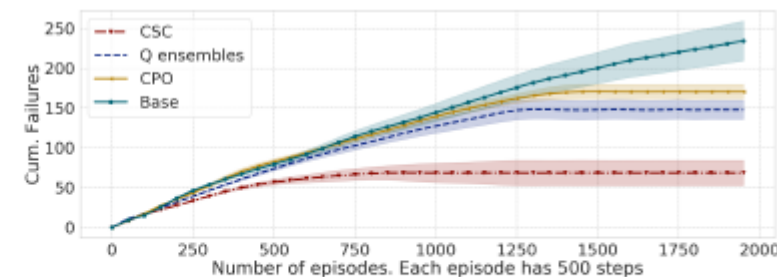
(c) Point 2D Nav. Cum. failures



(g) Car Rewards



(h) Car Avg. failures



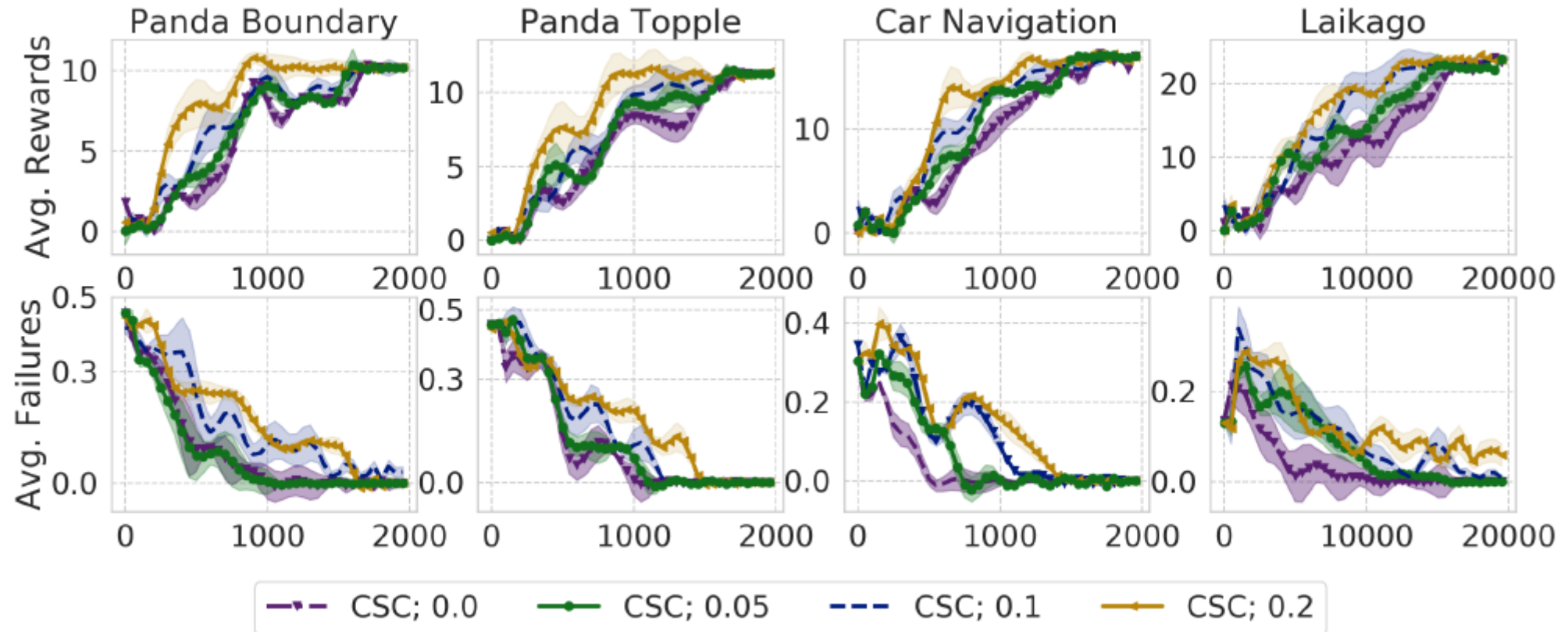
(i) Car Cum. failures

Comparable or better performance with significantly lower failures during training

* *base*: TRPO

4. Experiments

(3) Results



lower threshold, lower avg failures, lower reward convergence rate
few failures in initial stage of training due to high approximation of learned critic,
but quickly drop after about 500 episodes of training.

Conservative Safety Critics for Exploration

24.07.24

Kangyeon Kim



Control and
Intelligent Systems
Laboratory

Appendix

[Actor-Critic Algorithm]

batch actor-critic algorithm:

- 
1. sample $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ (run it on the robot)
 2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
 3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
 2. update $\hat{V}_\phi^\pi$ using target $r + \gamma \hat{V}_\phi^\pi(\mathbf{s}')$
 3. evaluate $\hat{A}^\pi(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}') - \hat{V}_\phi^\pi(\mathbf{s})$
 4. $\nabla_\theta J(\theta) \approx \nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) \hat{A}^\pi(\mathbf{s}, \mathbf{a})$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Appendix

[Proof of Thm 3.1 of CQL]

Proof of Theorem 3.1. In order to start, we first note that the form of the resulting Q-function iterate, $\hat{Q}^k$, in the setting without function approximation. By setting the derivative of Equation 1 to 0, we obtain the following expression for $\hat{Q}^{k+1}$ in terms of $\hat{Q}^k$,

$$\forall \mathbf{s}, \mathbf{a} \in \mathcal{D}, k, \quad \hat{Q}^{k+1}(\mathbf{s}, \mathbf{a}) = \hat{\mathcal{B}}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}) - \alpha \frac{\mu(\mathbf{a}|\mathbf{s})}{\hat{\pi}_\beta(\mathbf{a}|\mathbf{s})}. \quad (9)$$

Now, since, $\mu(\mathbf{a}|\mathbf{s}) > 0, \alpha > 0, \hat{\pi}_\beta(\mathbf{a}|\mathbf{s}) > 0$, we observe that at each iteration we underestimate the next Q-value iterate, i.e. $\hat{Q}^{k+1} \leq \hat{\mathcal{B}}^\pi \hat{Q}^k$.

Accounting for sampling error. Note that so far we have only shown that the Q-values are upper-bounded by the “empirical Bellman targets” given by, $\hat{\mathcal{B}}^\pi \hat{Q}^k$. In order to relate $\hat{Q}^k$ to the true Q-value iterate, Q^k , we need to relate the empirical Bellman operator, $\hat{\mathcal{B}}^\pi$ to the actual Bellman operator, $\mathcal{B}^\pi$. In Appendix D.3, we show that if the reward function $r(\mathbf{s}, \mathbf{a})$ and the transition function, $T(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ satisfy “concentration” properties, meaning that the difference between the observed reward sample, $r(\mathbf{s}, \mathbf{a}, r, \mathbf{s}') \in \mathcal{D}$ and the actual reward function $r(\mathbf{s}, \mathbf{a})$ (and analogously for the transition matrix) is bounded with high probability, then overestimation due to the empirical Backup operator is bounded. Formally, with high probability (w.h.p.) $\geq 1 - \delta, \delta \in (0, 1)$,

$$\forall Q, \mathbf{s}, \mathbf{a} \in \mathcal{D}, \quad \left| \hat{\mathcal{B}}^\pi Q(\mathbf{s}, \mathbf{a}) - \mathcal{B}^\pi Q(\mathbf{s}, \mathbf{a}) \right| \leq \frac{C_{r,T,\delta} R_{\max}}{(1 - \gamma) \sqrt{|\mathcal{D}(\mathbf{s}, \mathbf{a})|}}.$$